

INSTITUTO TECNOLÓGICO DE BUENOS AIRES

Introducción a la Computación 22.06

Display

Trabajo de Laboratorio N ° 5

Curso 2020A – 1er Cuatrimestre

GRUPO N°: 4		INTEGRANTES
Legajo N°	Nombre	
62490	Marcos Dario Alegre	
62012	Santiago Feldman	
62447	Anna Candela Gioia Perez	
61758	Juan Sebastián Helou	

	Fecha	Docente
Realizado	20/10/2020	
Presentado	21/10/2020	
Aprobado		

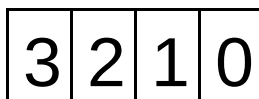
Introducción a la Computación

Trabajo práctico N° 5

Display

Manejo de un display alfanumérico

El simulador del Wookie cuenta con un display alfanumérico. El mismo se puede pensar como un array de 4 elementos, donde cada elemento es un dígito. Los elementos se numeran del 0 al 3 comenzando desde la derecha (ver esquema).



Para el manejo del display la cátedra creo una serie de funciones. Por lo general cuando se maneja un periférico se crean librerías que contienen los drivers del periférico, las funciones que se encuentran a continuación podrían considerarse como partes de una librería de manejo de display.

Un periférico antes de comenzar a utilizarse debe inicializarse. El display cuenta con una rutina de inicialización **initdis**. Antes de utilizar el display, por primera vez, se deberá invocar a esta subrutina.

```
initdis    equ    *  
  
    psha  
  
    ldaa #$FF    ;set for output  
    staa DDRC    ;C output  
    ldaa PIOC    ;get current value  
    anda #$FE    ;reset bit 0 to 0  
    staa PIOC    ;send it out  
  
    pula  
  
    rts
```

Una librería de manejo de periférico debe contar con funciones de entrada y salida. La cátedra les brinda la función **outchar** (**output character**), la misma se utiliza para enviar un carácter alfanumérico al display.

La función recibe en el registro A el carácter ASCII que se quiere mostrar en el display. El registro B debe contener la posición en la que se quiere colocar el carácter sobre el display (0 a 3). A continuación, se muestra el código de la función **outchar**.

```
*****
*Outchar: ACCA: Carácter a enviar
*          ACCB: Posición (0-3)
*****
outchar    equ    *

            stab PORTCL    ;set the position
            staa PORTB     ;set the character

            rts
```

Para que las funciones de la librería de manejo de display se puedan utilizar se deberán definir las siguientes contantes simbólicas en el código fuente.

```
DDRC    equ $1007    ;data direction register for C
PIOC     equ $1002    ;for strobe B assertion change
PORTCL   equ $1005    ;port C latched
PORTB    equ $1004    ;port B
```

Definición del problema

Se pide escribir las funciones necesarias para mostrar una frase en el display en forma secuencial (moviéndose de derecha a izquierda continuamente un carácter por vez). El intervalo de desplazamiento de cada carácter deberá estar comprendido entre 0,5 y 1 segundo (quedando a criterio del alumno el tiempo que mejore la visualización de la frase). Las frases deberán estar terminadas por el terminador cero.

Además de esto, el grupo deberá incluir en el código una constante simbólica (EQU) que al cambiarse de valor (ejemplo: pasar de 0 a 1) haga que la frase en del display se mueva en sentido contrario (de izquierda a derecha). Obviamente será necesario volver a compilar el código para que los cambios tomen efecto. Esta condición se evaluará en la entrega del TP.

Es recomendable familiarizarse con las funciones del display antes de comenzar a programar el desplazamiento de frases, escribir un programa de prueba que consista en colocar una letra en distintas posiciones del display es una buena manera de empezar este trabajo práctico.

Importante:

- Documentar claramente el intervalo de tiempo elegido y el cálculo del mismo.
- Armar un esqueleto para el programa. Definir funciones y un diagrama de flujo antes de empezar a programar. Modularizar el desarrollo al máximo.
- Utilizar el stack en forma eficiente.
- Es recomendable borrar el display antes de comenzar.
- Los mensajes deben definirse con las directivas FCC y FCB.
- Utilizar constantes simbólicas para clarificar el código.
- Seguir los criterios de buena programación.

Fecha de entrega: 21 de octubre de 2020

La entrega consta de 2 partes. Para considerarse aprobado el TP se deberá presentar el display funcionando en la clase y luego presentar el código. En la presentación el docente evaluará la participación de cada miembro en el desarrollo del trabajo practico así como el funcionamiento general del proyecto.

Presentación de los diagramas de flujo

El MAIN PROGRAM se dividió en 4 módulos, cada uno con una función distinta y con su respectivo diagrama de flujo. A continuación, insertamos el diagrama general del MAIN PROGRAM.

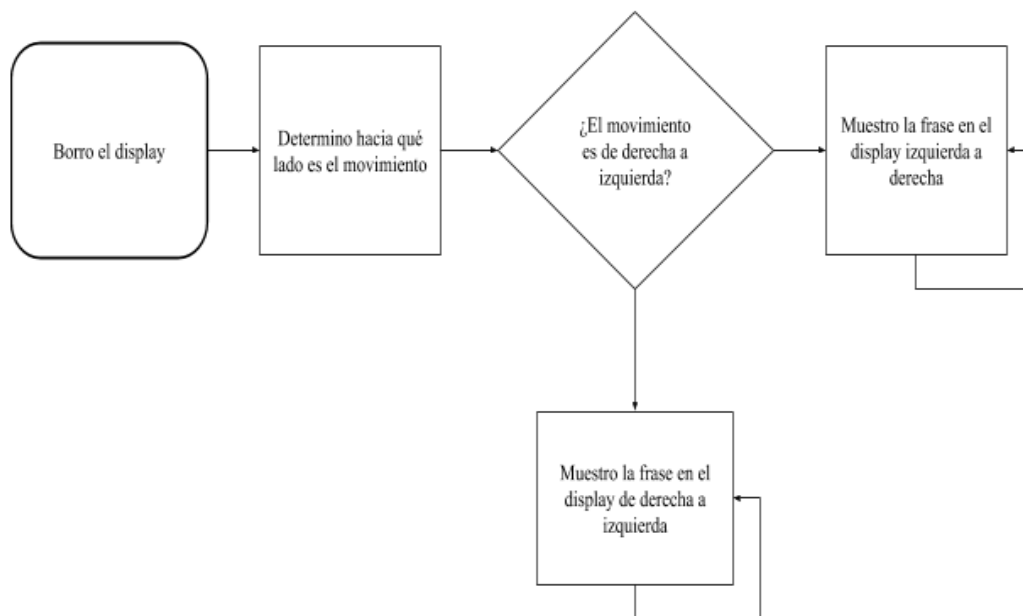


Diagrama 1.1: diagrama de flujo general del MAIN PROGRAM

El primer módulo limpia el display completamente así no se encuentran dificultades a la hora de mostrar la frase deseada. Al enviarle el valor “0” a las posiciones, el display borra lo que se encuentra en ellas, mostrándolas vacías. El esquema siguiente es el diagrama de flujo correspondiente a este primer módulo.

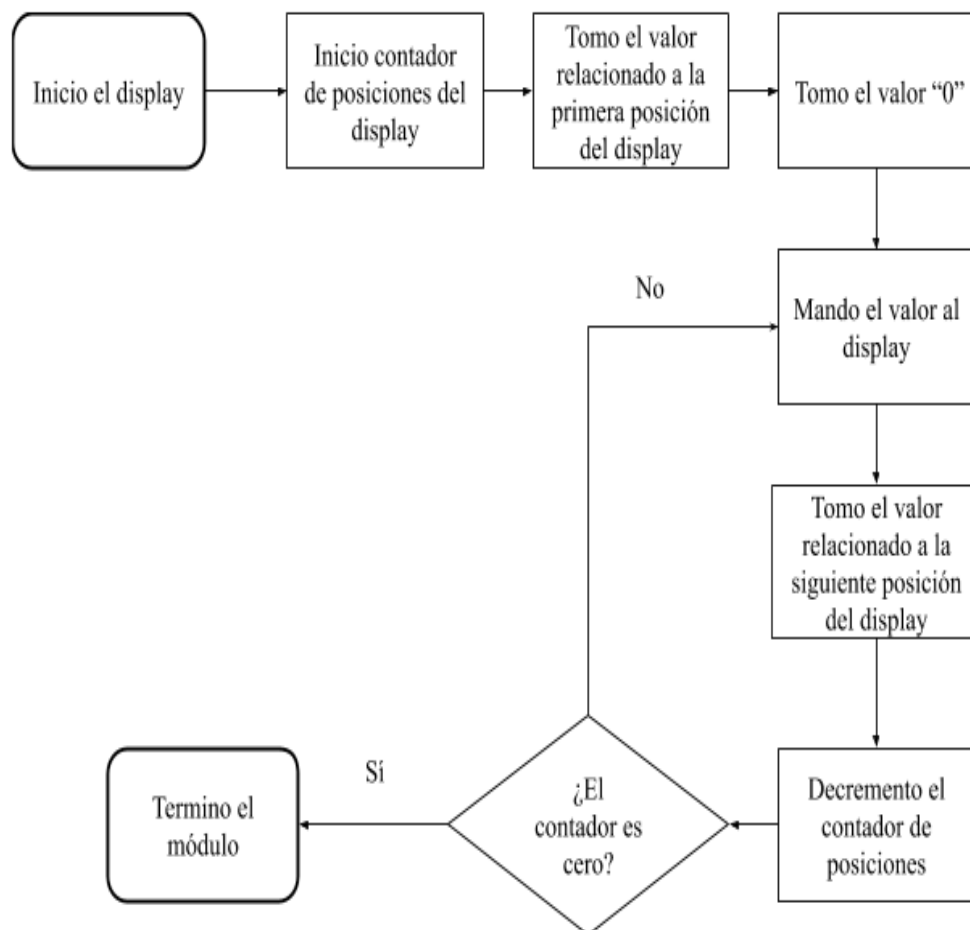


Diagrama 1.2: diagrama de flujo del primer módulo

El segundo módulo decide si el movimiento de la frase debe ser de derecha a izquierda o viceversa, dependiendo si la variable “rotar” es cero u otro número.

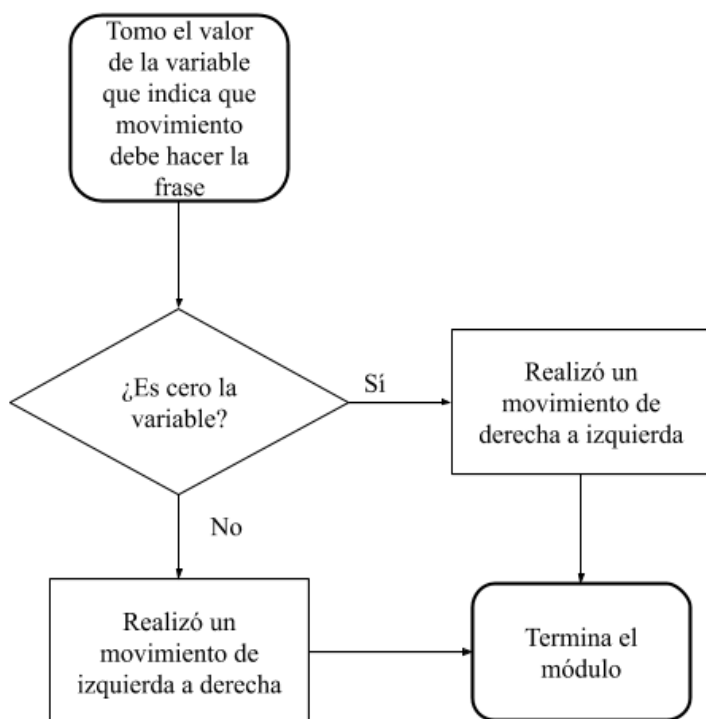


Diagrama 1.3: diagrama de flujo del segundo módulo

Tanto el tercer como el cuarto módulo llaman a la subrutina correspondiente la cantidad de veces necesarias para mostrar la frase en el display; pero, el tercer módulo corresponde al movimiento de derecha a izquierda y el cuarto al de izquierda a derecha. Los diagramas del tercer y cuarto módulo se presentan a continuación:

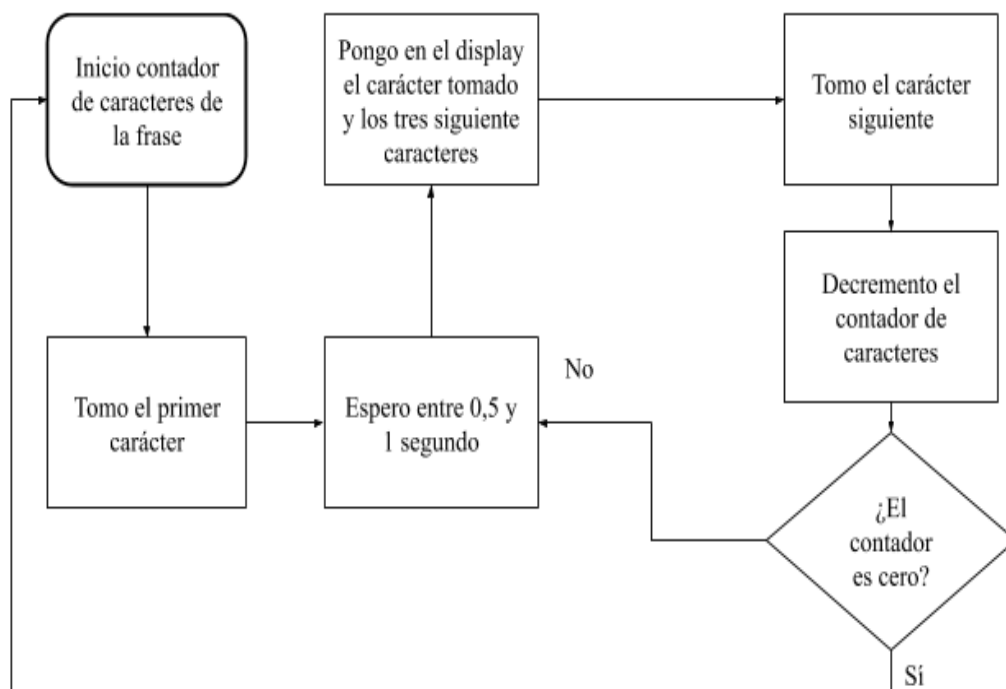


Diagrama 1.4: diagrama de flujo del tercer módulo

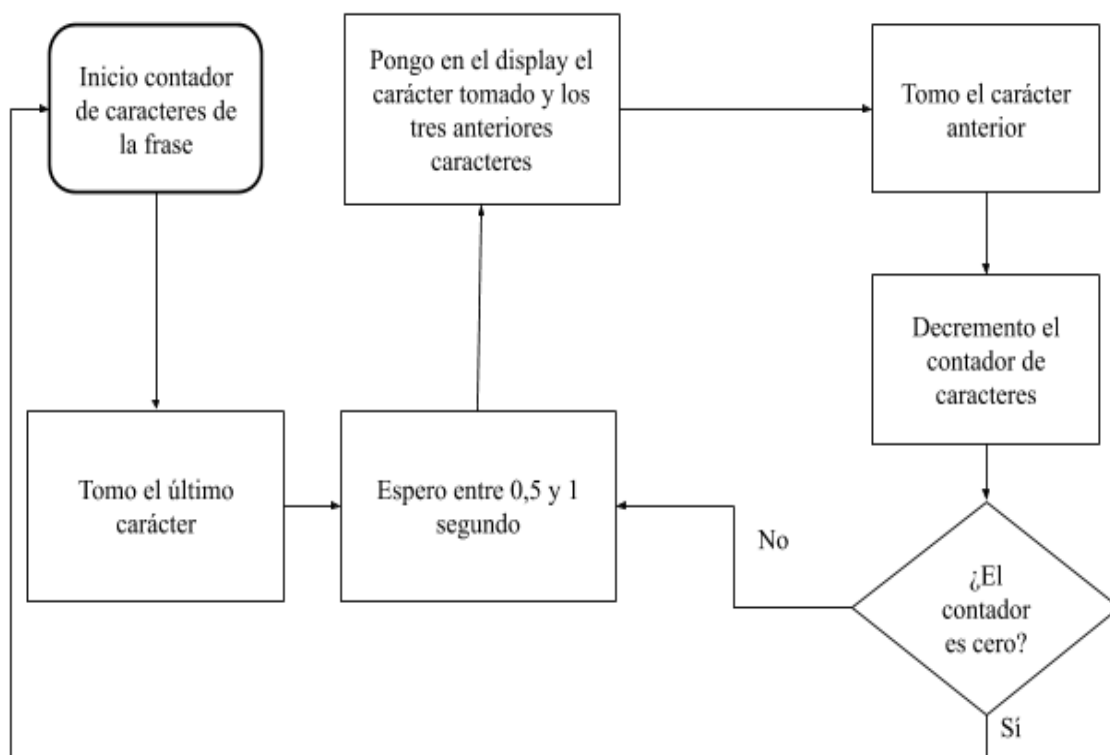


Diagrama 1.5: diagrama de flujo del cuarto módulo

Como se mencionó anteriormente, los últimos módulos llaman a distintas subrutinas encargadas de tomar ciertos caracteres y mostrarlos en el display dependiendo del movimiento. Si el movimiento es de derecha a izquierda, la subrutina se denomina SUBR1; si el movimiento es de izquierda a derecha, la subrutina correspondiente recibe el nombre de SUBR2. Los diagramas de flujo correspondientes se muestran a continuación:

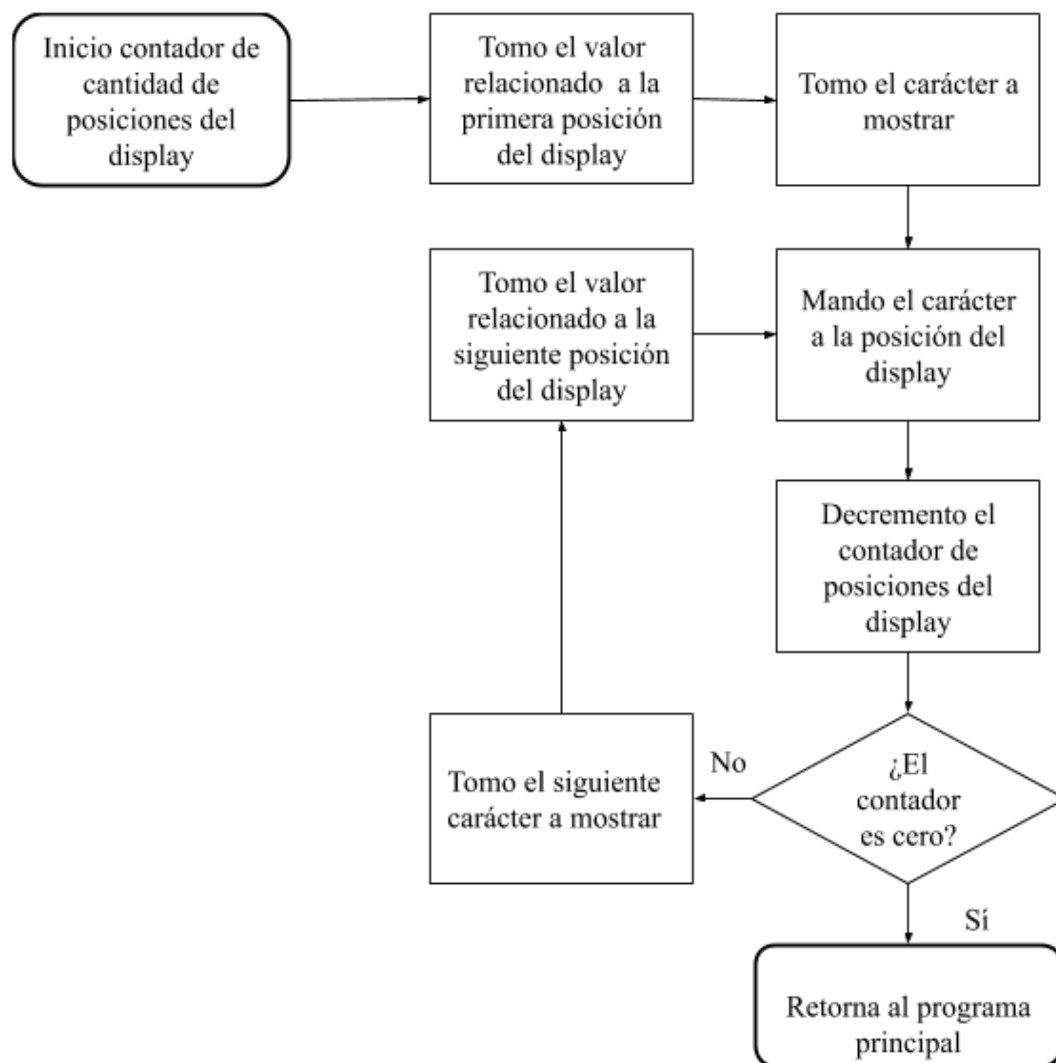


Diagrama 1.6: diagrama de flujo de la subrutina SUBR1

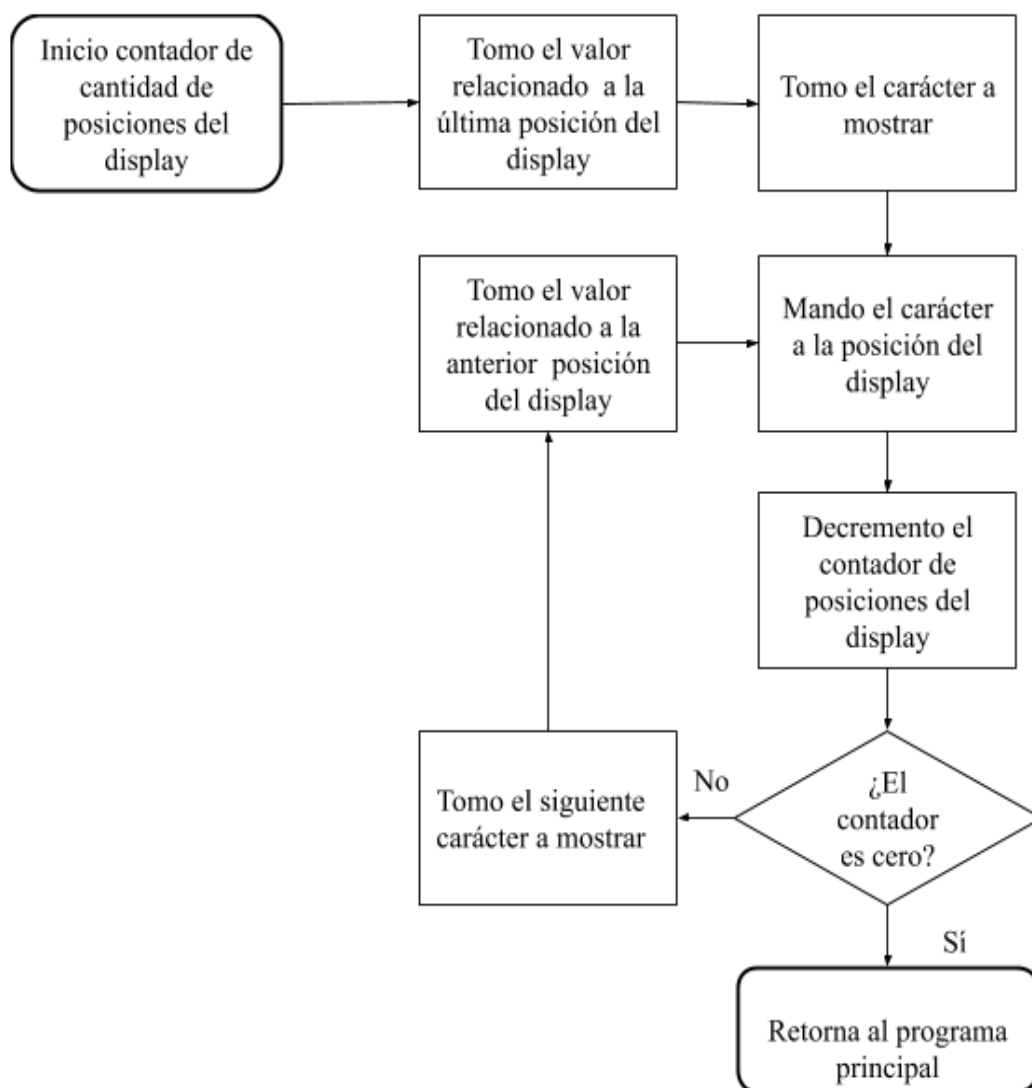


Diagrama 1.7: diagrama de flujo de la subrutina SUBR2

Presentación del código del programa

TP5J.LST

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           21/Oct/20  20:07:06  #
#
#   Source   =   tp5j.msa                               #
#   List     =   tp5j.lst                               #
#   Object   =   tp5j.r07                               #
#   Options  =                                         #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1 0000                                p68h11
2
3
4 0000                                extern  initdis,outchar,delay
5
6 0003          DI_START      EQU      $03
7 0000          DI_END        EQU      $00
8
9          *****
10         *   MAIN PROGRAM: se encarga de borrar lo que se encuentra      *
11         *   en el display, de ver hacia que lado es el desplazamiento    *
12         *   y de llamar a las subrutinas la cantidad de veces            *
13         *   necesarias para que se muestre la frase completa            *
14         *****
15
16 0000          rotar          EQU      0                ;variable que define hacia que lado se muestra la frase
17
18 0000                                rseg      CODE
19
20 0000 BD0000          jsr      initdis          ;necesario para iniciar el display
21
22 0003 C603            ldab     #DI_START        ;toma el valor de la primera posición del display
23 0005 CE0004          ldx      #4              ;inicia contador de posiciones del display
24 0008 8600          loop     ldaa     #0        ;toma el valor 0 para borrar el display
25 000A BD0000          jsr      outchar         ;llama a la subrutina para poner valores del display
26 000D 5A            decb     ;decrementa la posición del display
27 000E 09            dex      ;decrementa el contador de posiciones
28 000F 26F7          bne      loop             ;si el contador no es 0 repite la secuencia
29
30 0011 8E0063          lds      #stackp         ;necesario para el stack y la subrutina
31 0014 8600          ldaa     #rotar           ;toma el valor de la variable que indica el movimiento
32 0016 2703          beq      der              ;si es 0 la variable, muévase de derecha a izquierda
33 0018 7E0030          jmp      izq            ;si no es 0, el movimiento es de izquierda a derecha
34
35 001B 18CE0013  der      ldy      #long        ;inicia contador de longitud de la frase
36 001F CE0001          ldx      #msg           ;apunta al primer caracter
37 0022 BD0000          jsr      subr1          ;salta a la subrutina adecuada
38 0025 BD0000          jsr      delay         ;llama a la subrutina que genera el delay
39 0028 08            inx      ;incrementa el puntero de los caracteres
40 0029 1809          dey      ;decrementa el contador
41 002B 27EE          beq      der              ;si el contador es 0, repite el proceso
42 002D 7E0022          jmp      repeat         ;si el contador no es 0, repite la secuencias
```

```

43
44 0030 18CE0013  izq      ldy      #long      ;inicia contador de longitud de la frase
45 0034 CE0012      ldx      #msg2      ;apunta al último carácter
46 0037 BD001F      repeat2  jsr      subr2      ;salta a la subrutina adecuada
47 003A BD0000      jsr      delay      ;llama a la subrutina que genera el delay
48 003D 09          dex          ;decrementa el puntero de los caracteres
49 003E 1809        dey          ;decrementa el contador
50 0040 27EE        beq      izq      ;si el contador es 0, repite el proceso
51 0042 7E0037      jmp      repeat2  ;si el contador no es 0, repite la secuencias
52
53
54
55
56 *****
57 *  SUBROUTINA SUBR1: esta subrutina se encarga de  *
58 *  poner en el display 4 letras de la frase con un *
59 *  movimiento de derecha a izquierda              *
60 *****
61
62
63 0000      subr1      rseg      SUBR
64
65 0000 36      psha          ;hace backup de los registros
66 0001 37      pshb
67 0002 3C      pshx
68 0003 183C     pshy
69 0005 18CE0004 ldy      #4      ;inicia contador de caracteres a mostrar
70 0009 C603     ldab      #DI_START ;toma el valor de la última posición del display
71 000B A600     loop2     ldaa     0,x    ;toma el un carácter de la frase
72 000D BD0000     jsr      outchar    ;llama a la subrutina para poner valores del display
73 0010 1809     dey          ;decrementa contador de caracteres
74 0012 2705     beq      ter      ;si el contador es 0, termina la subrutina
75 0014 08       inx          ;si no es 0, incrementa el puntero de los caracteres
76 0015 5A       decb         ;decrementa la posición del display
77 0016 7E000B     jmp      loop2     ;vuelve a repetir la secuencia
78 0019 1838     ter          ;hace restore de los registros
79 001B 38       pulx
80 001C 33       pulb
81 001D 32       pula
82
83 001E 39       rts          ;retorna de la subrutina
84
85
86 *****
87 *  SUBROUTINA SUBR2: esta subrutina se encarga de  *
88 *  poner en el display 4 letras de la frase con un *
89 *  movimiento de izquierda a derecha              *
90 *****
91
92
93 001F      subr2      rseg      SUBR
94
95 001F 36      psha          ;hace backup de los registros
96 0020 37      pshb
97 0021 3C      pshx
98 0022 183C     pshy
99 0024 18CE0004 ldy      #4      ;inicia contador de caracteres a mostrar
100 0028 C600     loop3     ldab      #DI_END ;toma el valor de la última posición del display
101 002A A600     ldaa     0,x    ;toma el un carácter de la frase
102 002C BD0000     jsr      outchar    ;llama a la subrutina para poner valores del display
103 002F 1809     dey          ;decrementa contador de caracteres

```

```

104 0031 2705          beq      ter2          ;si el contador es 0, termina la subrutina
105 0033 09           dex          ;si no es 0, decrementa el puntero de los caracteres
106 0034 5C           incb         ;incrementa la posición del display
107 0035 7E002A        jmp      loop3        ;vuelve a repetir la secuencia
108 0038 1838          puly         ;hace restore de los registros
109 003A 38           pulx
110 003B 33           pulb
111 003C 32           pula
112
113 003D 39           rts              ;retorna de la subrutina
114
115
116 *****
117 *   ÁREA DE DATOS: Lista de caracteres ASCII que componenen *
118 *   la frase a mostrar en el display *
119 *****
120
121
122 0000              rseg      CONST
123
124 0000 00           fcb      0
125 0001 00          msg      fcb      0          ;necesarios para el comienzo de la muestra de la frase
126 0002 00           fcb      0
127 0003 00           fcb      0
128 0004 49544241     fcc      'ITBA GRUPO 4';define la frase que se quiere mostrar
129 0008 20475255
130 000C 504F2034
131 0010 00           fcb      0          ;terminador
132 0011 00           fcb      0          ;necesarios para el final de la muestra de la frase
133 0012 00          msg2     fcb      0
134 0013 00           fcb      0
135 0013          long      equ      *-msg      ;define la longitud de la frase con más parámetros
136
137 *****
138 *   ÁREA DE STACK: reserva las posiciones del stack *
139 *   con un margen extra *
140 *****
141
142 0000              rseg      STACK
143 0000          stack      rmb      100
144 0063          stackp     EQU      *-1
145
146
147 0064              END

Errors:  None          #####
Bytes:   151          # tp5j #
CRC:     2C98          #####

```

DELAY.LST

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           21/Oct/20  20:07:38  #
#
#   Source   =   delay.msa                                #
#   List     =   delay.lst                                #
#   Object   =   delay.r07                                #
#   Options  =                                           #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1 0000                p68h11
2
3
4
5 0000                public  delay
6
7
8
9 0008                L      EQU      $8
10 00FF               M      EQU      $FF
11 00DF               S      EQU      $DF
12
13
14
15 0000                rseg    SUBR
16
17 0000                delay  org      *
18 0000 36             psha
19 0001 37             pshb                ;se hace un backup de los registros a modificar
20 0002 3C             pshx
21
22 0003 CE0008         countl  ldx      #L      ;crea el contador mayor
23 0006 09             countl  dex
24 0007 270D           beq      endsrt1 ;si es cero el contador mayor va a terminar el programa
25
26 0009 86FF           countm  ldaa     #M      ;si no es cero reinicia el contador mediano
27 000B 4A             countm  decb
28 000C 27F8           beq      countl ;si es cero el contador mediano pasa a decrementar el mayor
29
30 000E C6DF           counts  ldab     #S      ;si no es cero reinicia el contador menor
31 0010 5A             counts  decb
32 0011 26FD           bne      counts
33 0013 7E000B        jmp      countm ;si es cero el contador menor decrementa el mediano
34
35 0016 38             endsrt1 pulx
36 0017 33             pulb                ;se recuperan los valores guardados
37 0018 32             pula
38 0019 39             rts
39
40 001A                END

Errors:  None          #####
Bytes:   26            # delay #
CRC:     3319          #####
```

Aclaración:

Se eligió el máximo intervalo de tiempo permitido para el delay (1 segundo) debido a que los ciclos de máquina del emulador en la práctica son mucho más rápidos que los ciclos teóricos de 0,5 μ s y queríamos que la frase fuera lo más legible posible.

La estructura de la subrutina del delay se divide en tres secciones dominadas cada una por su propio contador, para calcular qué valor había que asignarle a cada contador tuvimos primero que calcular cuántos ciclos de máquina debía usar la subrutina para llegar al segundo de tiempo. Como cada ciclo de máquina toma aproximadamente 0,5 μ s y cada segundo equivale a 1.000.000 μ s, llegamos a que debíamos utilizar 2.000.000 de ciclos de máquina en la subrutina.

Luego de tener una idea de cómo funcionaría el programa, se anotó el número de ciclos que tomaba cada instrucción para así hacer las cuentas y ver cuántas veces había que repetir cada sección de la subrutina (cantidades que se controlan a partir de los valores ingresados a los contadores).

Desde ahí, fue necesario entender cómo el valor de un contador afectaba al resto del programa para saber exactamente cuantas veces se repetiría cada instrucción. También se observó qué instrucciones se ejecutaban una sola vez sin importar qué modificaciones se le hicieran a los contadores.

Por último, se calibraron los valores a ingresar a cada contador para llegar a un número de ciclos totales que se aproximara a 2.000.000. La subrutina, desde el jsr hasta el rts, toma 2.000.384 ciclos de máquina en ejecutarse; se deja extremadamente cerca del segundo que era el objetivo (1,000192 segundos) si se considera que cada ciclo de máquina tarda exactamente 0,5 μ s en ejecutarse.

La cuenta queda: $43 + (\#L - 1)(8 + \#M * 5 + (\#M - 1)(5 + \#S * 5))$ si se toma desde el jsr hasta el rts.

DISP.LST

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           21/Oct/20  20:07:22  #
#
#   Source   =   disp.msa                               #
#   List     =   disp.lst                               #
#   Object   =   disp.r07                               #
#   Options  =                                           #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1  0000                                p68h11
2  0000                                rseg    SUBR
3
4  0000                                public initdis,outchar
5
6                                $display.def
7  1007                                DDRC      equ    $1007            ;data direction register for C
8  1002                                PIOC      equ    $1002            ;for strobe B assertion change
9  1005                                PORTCL    equ    $1005            ;port C latched
10 1004                                PORTB     equ    $1004            ;port B
11
12 0000                                initdis   equ    *
13
14 0000 36                                psha
15
16 0001 86FF                                ldaa    #$FF                ;set for output
17 0003 B71007                            staa    DDRC                ;C output
18 0006 B61002                            ldaa    PIOC                ;get current value
19 0009 84FE                                anda    #$FE                ;reset bit 0 to 0
20 000B B71002                            staa    PIOC                ;send it out
21
```

```
22 000E 32          pula
23
24 000F 39          rts
25
26                *****
27                * Outchar: ACCA: Carácter a enviar
28                * ACCB: Posición (0-3)
29                *****
30
31 0010          outchar      equ      *
32
33 0010 F71005      stab      PORTCL      ;set the position
34 0013 B71004      staa      PORTB       ;set the character
35
36 0016 39          rts
37 0017          END
```

```
Errors:  None      #####
Bytes:   23        # disp #
CRC:     4182      #####
```

display.def

```
DDRC      equ      $1007      ;data direction register for C
PIOC      equ      $1002      ;for strobe B assertion change
PORTCL    equ      $1005      ;port C latched
PORTB     equ      $1004      ;port B
```

LI11.XCL

```
-ZCODE=2000          -! Code segment -!
-ZVARS=3000
-ZCONST=4000
-ZSUBR=5000
-ZSTACK=5500        -! Data segment -!
-c68hc11             -! Processor -!
-Fmotorola           -! Output Format -!
-x                   -! Cross Reference -!
-B                   -! Forced dump -!
```

LINKOUT.LST

```
#####  
#  
#      Micro Series Universal Linker V4.46C/DXT      21/Oct/120  20:07:53  #  
#  
#      Target CPU      = 68hc11      #  
#      List file       = linkout.lst  #  
#      Output file 1   = aout.a07     #  
#      Output format  = motorola      #  
#      Command line    = -f li11 (-ZCODE=2000 -ZVARS=3000 -ZCONST=4000  #  
#                        -ZSUBR=5000 -ZSTACK=5500 -c68hc11 -Fmotorola -x  #  
#                        -B) -A TP5J DELAY disp -l linkout.lst  #  
#  
#                        (c) Copyright IAR Systems 1993 #  
#####
```

```
*****  
*                                *  
*      CROSS REFERENCE          *  
*                                *  
*****
```

There are no program entries.

```
*****  
*                                *  
*      MODULE MAP              *  
*                                *  
*****
```

FILE NAME : tp5j.r07
PROGRAM MODULE, NAME : tp5j

SEGMENTS IN THE MODULE

=====

CODE

Relative segment, address : 2000 - 2044

SUBR

Relative segment, address : 5000 - 503D

CONST

Relative segment, address : 4000 - 4013

STACK

Relative segment, address : 5500 - 5563

FILE NAME : delay.r07
PROGRAM MODULE, NAME : delay

SEGMENTS IN THE MODULE
=====

SUBR
Relative segment, address : 503E - 5057
ENTRIES ADDRESS REF BY MODULE
delay 503E tp5j

FILE NAME : disp.r07
PROGRAM MODULE, NAME : disp

SEGMENTS IN THE MODULE
=====

SUBR
Relative segment, address : 5058 - 506E
ENTRIES ADDRESS REF BY MODULE
initdis 5058 tp5j
outchar 5068 tp5j

*
* SEGMENTS IN DUMP ORDER *
*

SEGMENT =====	START ADDRESS =====	END ADDRESS =====	TYPE =====	ORG =====	P/N =====	ALIGN =====
CODE	2000	- 2044	rel	stc	pos	0
VARs	Not in use		dse	stc	pos	0
CONST	4000	- 4013	rel	stc	pos	0
SUBR	5000	- 506E	rel	stc	pos	0
STACK	5500	- 5563	rel	stc	pos	0

*
* END OF CROSS REFERENCE *
*

Errors: none
Warnings: none